FILMORA

MOVIE STREAMING PLATFORM
README & STEP-BY-STEP DEVELOPMENT GUIDE

React • Node.js • Express • MongoDB • TMDB • YouTube • Cloudflare Stream • Payments

A practical build guide for a subscription-based streaming website

Prepared: 10 November 2026

Purpose of this guide
Use this document to understand the development order, prepare your computer, build the trailer catalogue, add subscriptions, protect full-movie playback, test the system and deploy it. The exact folder structure and commands may be adjusted after the existing project ZIP is inspected.

Important: this guide is a project plan, not a claim that the website has already been built or that third-party commercial permissions have been granted.

Contents

1. 1. Project overview and user journeys
2. 2. Technology and system architecture
3. 3. Prepare your Windows computer
4. 4. Recommended project structure
5. 5. Build the TMDB movie and trailer catalogue
6. 6. Database design
7. 7. Accounts and subscription payments
8. 8. Upload and stream full movies
9. 9. Admin dashboard
10. 10. Security, legal and privacy checklist
11. 11. Testing and deployment
12. 12. Troubleshooting and next steps

How to use the guide

Follow the phases in order. First make the basic website run locally. Next connect TMDB and make trailers work. Add accounts and the database after the catalogue is stable. Payment and protected full-movie playback come later, because they depend on secure backend logic and verified third-party accounts.

Do not paste real API keys, passwords, payment secrets or Cloudflare tokens into screenshots, GitHub, frontend code or this PDF. Use private environment variables on the backend.

1. Project overview

Filmora is planned as a subscription video-on-demand (SVOD) service. Visitors can browse movie information and watch available trailers. Paying members can watch full movies only when Filmora has valid distribution rights for the viewer's territory.

Area	What it does
Public catalogue	Displays movie titles, descriptions, genres, release dates and artwork, subject to TMDB permissions.
Trailer playback	Uses a trailer reference returned by TMDB, commonly a YouTube video key, and embeds the supported player.
Member accounts	Allows customers to register, sign in, manage profiles and view subscription status.
Subscriptions	Processes payments through an eligible payment provider and activates membership only after server-side verification.
Full movies	Streams Filmora-licensed content through a video delivery service such as Cloudflare Stream.
Admin	Allows authorized staff to manage licensed titles, uploads, customers, plans and reports.

Main user journey

1. A visitor opens Filmora and browses or searches the catalogue.
2. The visitor opens a movie page and watches an available trailer.
3. To watch the full movie, the visitor creates an account and chooses a subscription plan.

4 The payment provider processes the payment and sends a server-to-server notification (webhook).

5 Filmora verifies the transaction and activates the subscription.

6 When the member presses Play, Filmora checks subscription status, title availability, territory rights and device limits before issuing a short-lived playback token.

2. Technology and system architecture

Technology	Role	Important note
React + Vite	Website interface	Runs in the browser; never store private API secrets here.
Node.js + Express	Backend API	Owns authentication, permissions, payments and protected playback.
MongoDB + Mongoose	Application database	Stores Filmora users, plans, subscriptions, payments and licensed movie records.
TMDB API	Movie discovery metadata	Commercial use requires the appropriate permission or agreement from TMDB.
YouTube embedded player	Trailer playback	Embed the trailer using its video key when an appropriate video is available.
Cloudflare Stream	Full-movie video hosting and delivery	Use only for titles Filmora is legally allowed to distribute.
Payment provider	Checkout and payment processing	Confirm Rwanda business eligibility, supported countries, fees and recurring methods.

High-level architecture

```
Visitor / Member / Admin\n        |\n        v\nReact frontend (public pages, member area, admin UI)\
n        | HTTPS API requests\n        v\nNode.js + Express backend\n  |          |           |\n  v
        v           v\nMongoDB     TMDB API    Payment provider\n  |                  |\n  |
            signed webhook\n  |                      |\n  +------ Backend verifies payment\n
                |\n                        v\n                Subscription status\n
          |\n                       v\n               Cloudflare Stream\n                protect
ed playback
```

The browser should call your backend for private operations. Your backend calls TMDB and payment services using secrets held in environment variables. Public TMDB image URLs or trailer embeds may be rendered by the frontend where permitted, but private keys must not be sent to it.

3. Prepare your Windows computer

Install these tools before coding:

- **Node.js LTS** from https://nodejs.org/ — includes Node and npm.
- **Visual Studio Code** from https://code.visualstudio.com/ — code editor.
- **Git** from https://git-scm.com/ — version control.
- **MongoDB Atlas** or a local MongoDB installation — database.
- **A modern browser** such as Chrome or Firefox.

Check the installations in Windows PowerShell

```
node -v\nnpm -v\ngit --version
```

Each command should print a version number. If PowerShell blocks npm.ps1, try running npm commands through Command Prompt (cmd) or use the npm.cmd executable. Do not change computer security settings without understanding the effect.

Open your project folder in VS Code. In PowerShell, use cd to move into the actual folder. Example:

```
cd "$HOME\Desktop\filmora"
```

If you already have a ZIP project, extract it first and open the folder that contains the project's package.json. Do not run npm install from your Windows home folder or from an unrelated parent directory.

4. Recommended project structure

```
filmora/\n  frontend/\n    public/\n    src/\n      components/\n      pages/\n      services/\n
 context/\n      App.jsx\n      main.jsx\n    package.json\n    .env.example\n  backend/\n    config/
\n    controllers/\n    middleware/\n    models/\n    routes/\n    services/\n    webhooks/\n    serv
er.js\n    package.json\n    .env.example\n  .gitignore\n  README.md
```

This is a recommended structure. If your ZIP already contains a different working structure, inspect it before moving or deleting files.

Create the frontend (only if you are starting from an empty project)

```
npm create vite@latest frontend -- --template react\ncd frontend\nnpm install\nnpm install react-rout
er-dom
```

Create the backend (from the filmora project root)

```
mkdir backend\ncd backend\nnpm init -y\nnpm install express mongoose dotenv cors helmet express-rate-
limit bcryptjs jsonwebtoken\nnpm install --save-dev nodemon
```

In backend/package.json, add scripts similar to these (preserve other existing fields):

```
"scripts": {\n  "start": "node server.js",\n  "dev": "nodemon server.js"\n}
```

Run the frontend and backend in two separate terminal windows. The exact start commands depend on the package.json scripts in your project. A Vite frontend normally uses npm run dev; the backend example above uses npm run dev.

5. Build the TMDB movie and trailer catalogue

Before using TMDB in a paid business, contact TMDB and obtain the correct commercial-use permission or agreement. Do not assume the free API terms cover a subscription streaming service. Ask specifically about commercial use of metadata, images, video records and trailer references in a worldwide paid service.

TMDB setup checklist

1. Create or sign in to a TMDB account at https://www.themoviedb.org/.
2. Read the current API documentation and terms at https://developer.themoviedb.org/.
3. Request the API credentials that apply to your use case.
4. Contact TMDB about commercial licensing before launching or relying on the data commercially.
5. Store the TMDB API key on the backend in an environment variable.
6. Build backend routes that search movies and retrieve a selected movie's details and video records.
7. Display a trailer only when a suitable video record exists and embedding is permitted.

Example API endpoints to study

```
GET /3/search/movie?query=movie-title\nGET /3/movie/{movie_id}\nGET /3/movie/{movie_id}/videos
```

The movie videos endpoint can return trailer and other video records. Your application should inspect fields such as site, key, type, official and language where available. For a YouTube result, the key is the video identifier used by the embed URL.

Example React embed (illustrative only)

```
const trailerUrl = `https://www.youtube.com/embed/${trailerKey}`;\n\n<iframe\n  src={trailerUrl}\n  t
itle="Movie trailer"\n  allow="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope;
picture-in-picture; web-share"\n  allowFullScreen\n/>
```

Do not hard-code a made-up trailer key. Show a clear “Trailer unavailable” message when TMDB has no suitable trailer. Some videos may be region-restricted, removed, private or disallow embedding.

TMDB attribution

TMDB's current attribution and branding instructions must be followed for the approved use. Include the required notice and logo/credit treatment in the place and format TMDB specifies. Example notice commonly required by TMDB: “This product uses the TMDB API but is not endorsed or certified by TMDB.” Confirm the current wording and placement with the applicable terms.

6. Database design

Use MongoDB collections with Mongoose schemas. Store dates as Date values, amounts as numeric values with an explicit currency, and references as ObjectIds. Add validation and indexes. Never store a user's plain-text password.

Collection	Typical fields	Purpose
users	name, email, passwordHash, role, createdAt	Customer and administrator accounts.
movies	title, description, tmdbId, posterPath, trailerKey, streamVideoId, status, licenseTerritories, licenseExpiresAt	Movie metadata plus Filmora-owned upload and rights details.
plans	name, amount, currency, billingInterval, screenLimit, active	Monthly subscription products.
subscriptions	userId, planId, provider, providerCustomerId, status, startDate, nextBillingDate, cancelAtPeriodEnd	Membership state and billing period.
payments	userId, subscriptionId, providerReference, amount, currency, status, paidAt	Payment records and reconciliation.
watchHistory	userId, movieId, progressSeconds, updatedAt	Continue-watching and viewing history.
deviceSessions	userId, deviceId, lastSeenAt, revokedAt	Enforce permitted simultaneous sessions.
webhookEvents	provider, eventId, receivedAt, processedAt, status	Prevent duplicate payment event processing.

Important distinction

TMDB metadata and trailer references are not the same as full-movie streaming files. Keep your own Stream video ID, licensing status, permitted territories and license expiry in Filmora's database. Only publish full movies after rights and upload processing have been checked.

7. Accounts and subscription payments

Authentication sequence

1. Customer submits registration details over HTTPS.
2. Backend validates the data and hashes the password using a suitable password-hashing library.
3. Backend stores the user record and returns a secure session or token.
4. Login checks the password hash and applies rate limits.
5. Admin-only routes check the user's role on the backend.

Payment sequence

1. Customer selects a monthly plan.
2. Backend creates a checkout request with the payment provider.
3. Customer completes payment on the provider's secure checkout page.
4. Provider sends a signed webhook to Filmora's backend.
5. Backend validates the webhook signature and independently verifies the transaction with the provider.
6. Backend checks the payment reference, amount, currency and expected customer/plan.
7. Backend processes each event idempotently, then activates or renews the subscription.
8. If payment fails or a subscription expires, protected playback is denied according to the published terms.

Payment provider selection

Flutterwave is one provider to investigate for a Rwanda-based business, but confirm account approval, supported customer countries, settlement currency, fees, refunds, recurring card support and mobile-money renewal behavior directly with the provider. Do not assume every payment method supports automatic monthly renewal. Provide a clear manual-renewal process if needed.

Never activate a paid subscription only because the browser returns to a “success” page. A redirect is not proof of payment. The backend must verify the transaction.

8. Upload and stream full movies

Only upload films for which Filmora has valid distribution rights. Rights can be limited by country, platform, term, language, format or subscription model. A license for one territory does not automatically authorize worldwide streaming.

Recommended upload workflow

1. An administrator signs in and passes a backend role check.
2. Admin enters the movie title, description, language, runtime, category and rights information.
3. Backend requests a temporary direct-upload URL from Cloudflare Stream.
4. The browser uploads the video directly to Cloudflare using that temporary URL.
5. Backend stores the returned Stream video identifier and tracks processing status.
6. Admin confirms the video is ready, checks playback and captions, and publishes the title.
7. When a subscriber presses Play, backend checks the active subscription, movie status, territory rights, license expiry and device limit.
8. Backend issues a short-lived signed playback token or URL. The frontend player uses it to play the movie.

Protect full movies

- Keep Cloudflare API tokens on the backend only.
- Use signed playback tokens for paid content where supported.
- Use short token lifetimes and avoid exposing permanent private URLs.
- Check membership and territorial rights on every playback authorization.
- Apply device/concurrent-stream limits on the server.
- Do not assume signed URLs prevent all screen recording or piracy; they are one layer of protection.

9. Admin dashboard

Dashboard area	Functions
Overview	Registered users, active subscribers, monthly recurring revenue, failed payments, viewing activity and estimated streaming costs.
Movie management	Search/import permitted metadata, add rights details, upload licensed full movies, edit, publish/unpublish.
Customers	Search users, view account status, manage support requests and suspend accounts when justified.
Plans	Set plan name, price, currency, billing interval, screen limits and availability.
Payments	View provider references, verified payment status, refunds and reconciliation records.
Analytics	Popular titles, watch time, subscriber growth, cancellations and churn.
Settings	Site configuration, staff roles, audit logs and integration status.

Create separate roles such as customer, content_manager and admin. Do not rely on hiding dashboard buttons in React; enforce permissions in backend middleware.

10. Security, legal and privacy checklist

- Use HTTPS in production and secure, HttpOnly cookies where appropriate.

- Keep secrets in backend environment variables and out of Git.
- Hash passwords; validate inputs; use rate limiting and secure headers.
- Restrict CORS to the actual frontend domains.
- Verify payment webhooks and make webhook processing idempotent.
- Use least-privilege API tokens and rotate compromised credentials.
- Publish privacy, subscription, cancellation, refund and acceptable-use policies.
- Review data protection, consumer protection, tax, age-rating and payment obligations in every market you serve.
- Keep evidence of film distribution rights and their territories/expiry dates.
- Provide a support contact and a process for content-rights complaints.

11. Testing and deployment

Test locally before making the website public.

Test	Expected result
Homepage loads	No blank page or console-breaking error.
Movie search	Relevant results appear and empty searches are handled.
Movie details	Correct title, description, artwork and release information display.
Trailer available	The embedded player loads when the trailer is available and embeddable.
Trailer unavailable	A helpful message appears; the page does not crash.
Registration/login	Validation works; password is never stored in plain text.
Payment success	Only a verified provider transaction activates membership.
Duplicate webhook	Same event does not create duplicate payment or subscription records.
Expired subscription	Protected full-movie playback is denied.
Unlicensed territory	Playback is blocked where the title is not licensed.
Admin permissions	Normal customers cannot access admin APIs.
Mobile layout	Pages remain usable on phone and desktop.

Deployment order

1. Create production MongoDB database and a restricted database user.
2. Deploy the Node.js backend to a host that supports Node applications.
3. Set production backend environment variables in the host's secret manager.
4. Deploy the React frontend to a static frontend host.
5. Set the frontend API base URL and backend CORS allowlist.
6. Configure HTTPS and your domain name.
7. Set the payment provider's production webhook URL and verify test/live mode settings.
8. Configure Cloudflare Stream and test signed playback with a licensed test video.
9. Run the full test checklist and monitor errors, payments and streaming usage.

Example backend .env.example

```
PORT=5000\nNODE_ENV=development\nMONGODB_URI=replace_with_your_database_uri\nJWT_SECRET=replace_with_a_long_random_secret\nTMDB_API_KEY=replace_with_your_tmdb_key\nCLOUDFLARE_ACCOUNT_ID=replace_with_account_id\nCLOUDFLARE_STREAM_TOKEN=replace_with_server_token\nFLW_SECRET_KEY=replace_with_payment_secret\nFRONTEND_URL=http://localhost:5173
```

This is an example template only. Use real values only in your private local .env file or your hosting provider's secret settings. Add .env to .gitignore. Never upload it to GitHub.

12. Troubleshooting and next steps

Problem	What to check
npm is not recognized	Install Node.js LTS, reopen terminal and run node -v and npm -v.
npm.ps1 execution policy error	Use Command Prompt or npm.cmd in PowerShell; do not paste random policy commands.
Missing script: dev	Open package.json and inspect its scripts. Run a script that actually exists or add the correct script.
EJSONPARSE package.json	Fix invalid JSON: use double quotes, remove comments/trailing commas and check brackets.
MongoDB connection fails	Check URI, username/password encoding, network access rules and database service status.
TMDB returns unauthorized	Check the backend key, API authentication method, account/API access and request URL.
Trailer does not play	Check video site/key, embed permission, region availability, browser console and YouTube restrictions.
Payment says success but membership inactive	Inspect webhook delivery, signature verification, provider transaction verification and server logs.
Full movie does not play	Check Stream processing status, signed token expiry, allowed origin and subscription/rights checks.

Your first practical milestone

Do not try to complete every feature at once. The first milestone is a working Filmora homepage that can search TMDB movies, open a movie-details page and play an available trailer in an embedded player. After this works, continue to MongoDB and authentication, then payments, then full-movie streaming and admin tools.

What to prepare before the first coding session

- Your existing project ZIP (if you already started the website).
- A Windows computer with Node.js, VS Code and Git installed.
- A TMDB account and a clear request for commercial-use approval.
- A list of movies you have the legal right to stream in your intended territories.
- A plan for the business name and domain; check trademark and domain availability before investing in branding.

End of guide — Build in phases, test every phase and keep all credentials private. The actual code and setup steps should be tailored to the contents of your project ZIP before files are changed.